

Engineering manual

Unitree L1

Docker, RViz2 and OctoMap

ROS 2 Humble runtime

Exact environment reconstruction
Manual component-by-component operation
Stationary mapping and mobile-SLAM boundary
Three technical parts

Project: /home/isr/unitree_l1_project
Target: ros@<container>:/workspace/ros2_ws
Assembled: 23 July 2026

ENGINEER'S OPERATING PRINCIPLE

One physical LiDAR. One driver. One Docker runtime.
Additional terminals attach to that same runtime with docker exec.
ROS 2, RViz2 and OctoMap execute inside Ubuntu 22.04 Docker.

This is the reconstruction and operation manual for the environment that was actually built on the project computer. It is meant to let an engineer understand, rebuild, start, inspect and stop every layer manually.

The document has exactly three technical parts:

1. Unitree L1 configuration and Docker environment;
2. RViz2 configuration and use;
3. OctoMap configuration, mapping, saving and engineering evaluation.

The complete chronological command trace remains in docs/configuration-log.md and in the command-journal PDF. This manual turns that evidence into a reproducible procedure.

How to read the commands

Every command block has an execution boundary:

- HOST — isr@...\$: run on Ubuntu 24.04, outside Docker.
- DOCKER — ros@...:/workspace/ros2_ws\$: run inside the one project container.
- EVIDENCE — EXECUTED: a command class that was actually used during construction or validation.
- EXPECTED: output or state that must be checked; it is not a command.

Commands are not interchangeable across those boundaries. In particular, ros2, rviz2, colcon, the Unitree driver and octomap_server are Docker commands. Docker creation and device/X11 discovery are host commands.

Current operational warning

The final read-only audit on 23 July found three project containers on ROS_DOMAIN_ID=42. Two contained duplicate L1/OctoMap graphs, and the cloud was not producing samples. The corrected launch file itself passed its tests and a hardware-free one-RViz launch probe, but a new live pass must begin from one clean runtime.

Before the laboratory sequence in this manual:

1. press Ctrl+C once in every old launch terminal;
2. wait for the processes to finish;
3. type exit in every old ros@... shell;
4. verify from the host that no project container remains;
5. create exactly one new GUI/LiDAR container.

Do not open a second physical-LiDAR container. Open additional shells in the same container with docker exec.

Part 1 — Unitree L1 configuration and Docker environment

1.1 The engineering boundary

The host was not converted into a ROS 2 machine. Docker was already installed and operational; ROS 2 Humble was deliberately not installed on Ubuntu 24.04. The resulting separation is:

```
Unitree L1
|   TTL UART, 3.3 V logic, 2,000,000 bit/s
v
Unitree adapter / Silicon Labs CP2104
|
v
Ubuntu 24.04 host: /dev/ttyUSBx
|   selected Docker device mapping + exact tty GID
v
Ubuntu 22.04 / ROS 2 Humble container: /dev/unitree_lidar
|
v
unitree_lidar_ros2_node
+-- /unilidar/cloud   [sensor_msgs/msg/PointCloud2]
+-- /unilidar/imu     [sensor_msgs/msg/Imu]
|
+-- ll_monitor --> /diagnostics
```

The Ubuntu 24.04 host supplies only:

- the Docker Engine and client;
- the physical serial character device;
- the project files;
- the X11 display server;
- the GPU/DRI devices used to draw RViz.

The container supplies:

- Ubuntu 22.04 Jammy;
- ROS 2 Humble;
- colcon, rosdep and vcs;
- Unitree SDK and ROS 2 driver;
- PCL, TF2, RViz2;
- octomap_server and its dependencies.

An RViz window appearing on the Ubuntu desktop does not mean RViz runs on the host. The executable and ROS node remain in Docker; X11 carries the drawing protocol to the host display.

1.2 Why Ubuntu 22.04/Humble is in Docker

The initial audit found:

Host OS	Ubuntu 24.04.2 LTS, x86_64
Kernel	6.17.0-35-generic
Docker	Engine 29.6.1
Compose	5.3.1
Host ros2	absent
Host rviz2	absent
Host colcon	absent

ROS 2 Humble targets Ubuntu 22.04. Installing its Jammy binary packages into Ubuntu 24.04 Noble would mix incompatible distribution assumptions. The project therefore pins a Jammy/Humble image and leaves the host clean.

HOST — EVIDENCE, EXECUTED DURING INITIAL AUDIT

```
lsb_release -a
uname -a
lscpu
free -h
df -hT
lspci -nnk
lsusb
docker version
docker info
```

```
docker compose version
command -v cmake colcon ros2 rviz2
```

Purpose: establish what existed before configuration. Side effect: none; these are read-only inventory commands.
Interpretation: Docker existed; native ROS 2 Humble did not.

1.3 Locked identities and reproducibility

The principal runtime identities are:

```
Docker base
ros:humble-ros-base-jammy
sha256:5c793b92e0b12d6babb438cb20eed7766495fde6419a21e3d2e918464f09dc17

Unitree SDK
tag:      v1.0.16
commit: 1bd7d95d8ab7ce7a22058d2bb07e39fd62612aa6

OctoMap ROS stack
tag:      2.3.1
commit: f79da9a9alfcdf82e72dab4df288d6cc27c6e163

Verified container components
Ubuntu 22.04
ROS 2 Humble
GCC 11.4.0
CMake 3.22.1
PCL 1.12.1
Eigen 3.4.0
Boost 1.74
liboctomap 1.9.7
octomap_msgs 2.0.1
octomap_ros 0.4.4
RViz2 11.2.27
```

The image digest and Git commits are pinned. Individual APT package versions were observed and recorded but are not all frozen in a snapshot repository; future rebuilding is controlled, not guaranteed byte-identical indefinitely. The authoritative version file is docs/versions-lock.md.

HOST — EVIDENCE, EXECUTED TO IDENTIFY THE BASE

```
docker buildx imagetools inspect ros:humble-ros-base-jammy
```

DOCKER — VERIFY PINNED SOURCE CHECKOUTS

```
cd /workspace/ros2_ws
git -C src/unilidar_sdk rev-parse HEAD
git -C src/unilidar_sdk describe --tags --exact-match HEAD
git -C src/octomap_mapping rev-parse HEAD
git -C src/octomap_mapping describe --tags --exact-match HEAD
```

Expected commits are the two 40-character identities listed above.

1.4 Hardware and electrical interface

Use this physical chain:

```
12 V / 1 A supply -----> Unitree L1 power
Unitree L1 3.3 V TTL UART --> Unitree adapter --> host USB
```

Important constraints:

- use the Unitree adapter;
- provide the separate 12 V / 1 A supply;
- USB 5 V is not the LiDAR power supply;
- the signal interface is TTL 3.3 V, not RS-232;
- keep the rotating mechanism clear;
- do not alter the 2,000,000 bit/s UART setting without measured evidence;
- stop software before unplugging the serial adapter.

The validated adapter identity was:

```
Product      Silicon Labs CP2104 USB-to-UART bridge
VID:PID      10c4:ea60
Serial       02C90122
Stable link   /dev/serial/by-id/
              usb-Silicon_Labs_CP2104_USB_to_UART_Bridge_
              Controller_02C90122-if00-port0
Resolved tty  /dev/ttyUSB0 during validation
```

The stable link should be used for discovery. /dev/ttyUSB0 may become /dev/ttyUSB1 after reconnection or when

another adapter exists.

HOST — IDENTIFY THE ADAPTER WITHOUT MODIFYING IT

```
cd /home/isr/unitree_l1_project
./scripts/check-lidar.sh
```

Or inspect the primitives:

```
lsusb
find /dev/serial/by-id -maxdepth 1 -type l -print
readlink -e \
  /dev/serial/by-id/PASTE_STABLE_UNITREE_LINK_NAME_HERE
udevadm info --query=property \
  --name=/dev/serial/by-id/PASTE_STABLE_UNITREE_LINK_NAME_HERE
```

Do not hardcode serial 02C90122 for a different adapter.

1.5 Project paths and bind mount

Compose maps the entire host project into the container:

```
HOST
/home/isr/unitree_l1_project

DOCKER
/workspace
  /workspace/ros2_ws
  /workspace/docker
  /workspace/scripts
  /workspace/docs
  /workspace/maps
  /workspace/bags
```

Therefore:

- build commands run from `/workspace/ros2_ws`;
- a file saved below `/workspace` persists on the host;
- source edits made on either side are immediately visible on the other;
- the desired prompt is `ros@<container>:/workspace/ros2_ws$`;
- `/workspace/ros2_ws/src/...` is source;
- `/workspace/ros2_ws/install/...` is the installed ROS overlay.

Do not confuse:

```
/workspace/ros2_ws/install/setup.bash    correct overlay
/workspace/ros2_ws/src/install/setup.bash  wrong path
```

`setup.bash` is sourced into the current shell; it is not executed:

```
source /workspace/ros2_ws/install/setup.bash
```

Never use:

```
./install/setup.bash
sudo ./install/setup.bash
```

1.6 What the Dockerfile configures

`docker/Dockerfile` starts from the pinned Humble/Jammy image and installs:

Build	build-essential, cmake, git, pkg-config
ROS tools	colcon, rosdep, vcs
LiDAR	PCL, Boost and Unitree build dependencies
Visual	rviz2, xauth, x11-utils, Mesa/DRI
TF/data	tf2_ros, pcl_conversions, pcl_ros, rosbag2
OctoMap	liboctomap-dev, octomap_msgs, octomap_ros

It creates non-root user `ros` with the host numeric UID/GID. That makes files created through the bind mount belong to the normal host user. The final image state is:

```
USER      ros
WORKDIR   /workspace/ros2_ws
ENTRYPOINT /workspace/docker/entrypoint.sh
```

The entrypoint sources:

```
source /opt/ros/humble/setup.bash
source /workspace/ros2_ws/install/setup.bash # if it exists
```

However, `docker exec` starts a new shell without re-running the image entrypoint. Every attached shell must source both setup files explicitly.

1.7 What each Compose layer configures

The configuration is split so access is explicit.

docker/compose.yaml — base runtime

```
image            unitree-l1:humble-v1.0.16
working directory /workspace/ros2_ws
bind mount       project root -> /workspace
ROS_DOMAIN_ID    42 by default
security         no-new-privileges, all capabilities dropped
user             non-root ros
```

ROS_DOMAIN_ID=42 isolates this ROS graph from default-domain processes. Every process that must communicate needs the same value.

docker/compose.gui.yaml — X11 and GPU

It passes only:

```
DISPLAY
XAUTHORITY -> /tmp/host.xauthority, read-only
/tmp/.X11-unix, read-only
/dev/dri
actual VIDEO_GID and RENDER_GID
QT_X11_NO_MITSHM=1
```

This project does not use xhost +, privileged: true, or blanket device access.

docker/compose.lidar.yaml — one serial device

It maps:

```
host selected tty --> /dev/unitree_lidar:rwm
```

and adds only that tty's numeric group ID. Inside Docker it defines:

```
LIDAR_PORT=/dev/unitree_lidar
```

The fixed internal name is independent of whether Linux chose ttyUSB0 or ttyUSB1.

1.8 Build the environment from a clean checkout

These commands reconstruct the image and workspace. Network access is needed for the image, APT packages and missing Git checkouts.

HOST — BUILD THE PINNED PROJECT IMAGE

```
cd /home/isr/unitree_l1_project
export HOST_UID="$(id -u)"
export HOST_GID="$(id -g)"

docker compose -f docker/compose.yaml build --pull
```

Project convenience command, which performs the same class of operation:

```
./scripts/docker-build.sh
```

Purpose: build unitree-l1:humble-v1.0.16. Side effect: downloads image/package layers and creates a local image.

Hardware: not required.

DOCKER — IMPORT THE PINNED SOURCE DEPENDENCIES

```
cd /workspace/ros2_ws
source /opt/ros/humble/setup.bash

vcs import --skip-existing src \
  < /workspace/config/dependencies.repos
```

config/dependencies.repos contains the exact Unitree and OctoMap commits. --skip-existing preserves already-present checkouts.

DOCKER — INSTALL DECLARED SYSTEM DEPENDENCIES

```
rosdep install \
  --from-paths \
    src/ll_bringup \
    src/ll_monitor \
    src/ll_octomap_bringup \
    src/octomap_mapping \
    src/unilidar_sdk/unitree_lidar_ros2 \
  --ignore-src \
  --rosdistro humble \
```

```
--skip-keys "ament_python pcl" \
-r -y
```

Purpose: install missing dependencies declared by ROS packages. Side effect: modifies only the disposable Docker image/container layer. Why skip keys: those dependencies are already deliberately supplied by the image and their rosdep resolution is not used here.

DOCKER — BUILD THE WORKSPACE

```
cd /workspace/ros2_ws
colcon list
colcon build
source install/setup.bash
```

The workspace-local colcon_defaults.yaml restricts package discovery to:

```
src/l1_bringup
src/l1_monitor
src/l1_octomap_bringup
src/octomap_mapping
src/unilidar_sdk/unitree_lidar_ros2
```

This is essential. The Unitree repository also contains a ROS 1/catkin package and a raw CMake SDK. Unrestricted discovery caused the photographed find_package(catkin) failure. The vendor checkout was not edited; the project corrected discovery at the workspace layer.

The defaults also enable --symlink-install. Python/config/launch source changes are therefore reflected conveniently, but a new installed filename or package metadata change still requires a rebuild and re-sourcing.

If the shell uses set -u, source ROS safely:

```
set +u
source /opt/ros/humble/setup.bash
source /workspace/ros2_ws/install/setup.bash
set -u
```

1.9 Create the desired ros@...:/workspace runtime

First clean old containers

HOST — READ-ONLY CHECK

```
docker ps \
--filter 'ancestor=unitree-l1:humble-v1.0.16' \
--format 'ID={{.ID}} NAME={{.Names}} STATUS={{.Status}}'
```

The clean starting condition is no output. If rows appear, return to the corresponding launch terminals, use Ctrl+C, wait, then exit. Do not kill an unknown process blindly.

Simplest audited entry command

HOST — CREATE ONE GUI/LIDAR CONTAINER

```
cd /home/isr/unitree_l1_project
./scripts/docker-shell.sh --gui --lidar
```

The wrapper is intentionally inspectable. It:

1. records the host UID/GID;
2. requires a readable X11 cookie and DRI devices;
3. finds exactly one stable serial adapter or refuses ambiguity;
4. resolves it to a character device;
5. accepts only ttyUSB* or ttyACM*;
6. refuses a tty already held by a process;
7. computes the tty/video/render GIDs;
8. merges base, GUI and LiDAR Compose files;
9. verifies the Compose model;
10. opens a non-root Bash shell at /workspace/ros2_ws.

Expected prompt:

```
ros@<container-id>:/workspace/ros2_ws$
```

Fully expanded manual equivalent

Use the actual stable by-id link discovered on this computer.

HOST — PREPARE ENVIRONMENT

```
cd /home/isr/unitree_l1_project

export HOST_UID="$(id -u)"
export HOST_GID="$(id -g)"
export ROS_DOMAIN_ID=42

export LIDAR_DEVICE="$(
  readlink -e \
    /dev/serial/by-id/PASTE_STABLE_UNITREE_LINK_NAME_HERE
)"

test -c "$LIDAR_DEVICE"
fuser -v "$LIDAR_DEVICE" || true
export LIDAR_GID="$(stat -Lc '%g' "$LIDAR_DEVICE")"

test -n "$DISPLAY"
test -n "$XAUTHORITY"
test -r "$XAUTHORITY"
test -c /dev/dri/card0
test -c /dev/dri/renderD128
export VIDEO_GID="$(stat -Lc '%g' /dev/dri/card0)"
export RENDER_GID="$(stat -Lc '%g' /dev/dri/renderD128)"
```

Run this from a terminal opened in the graphical Ubuntu session. Do not print the Xauthority cookie contents.

HOST — INSPECT THE MERGED MODEL

```
docker compose \
  -f docker/compose.yaml \
  -f docker/compose.gui.yaml \
  -f docker/compose.lidar.yaml \
  config
```

Read the resulting device, volume, environment, GID and security settings. Only then create the container:

HOST — CREATE ONE NAMED MANUAL RUNTIME

```
docker compose \
  -f docker/compose.yaml \
  -f docker/compose.gui.yaml \
  -f docker/compose.lidar.yaml \
  run --rm \
  --name unitree_l1_manual \
  -e REQUIRE_GUI=true \
  dev bash
```

This form makes later attachment simple:

```
docker exec -it unitree_l1_manual bash
```

1.10 Open several terminals in the same container

The physical L1 must have one driver and one owning container. A manual component-by-component experiment needs several shells, not several containers.

DOCKER — FIND THE CURRENT CONTAINER ID

```
hostname
```

HOST — ATTACH ANOTHER SHELL TO THAT EXACT CONTAINER

```
docker exec -it CONTAINER_ID_OR_NAME bash
```

EVERY NEW DOCKER SHELL — INITIALISE ROS

```
cd /workspace/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
```

Check that all shells agree:

```
printf 'host=%s domain=%s pwd=%s\n' \
  "$(hostname)" "$ROS_DOMAIN_ID" "$PWD"
```

1.11 Prove the runtime boundary and device access

DOCKER — REQUIRED PRE-FLIGHT

```
cd /workspace/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash

REQUIRED_GUI=true /workspace/scripts/assert-ros-container.sh

cat /etc/os-release
printf 'ROS_DISTRO=%s ROS_DOMAIN_ID=%s\n' \
  "$ROS_DISTRO" "$ROS_DOMAIN_ID"
command -v ros2
command -v rviz2
id
```

Expected guard marker:

```
DOCKER_ROS_RUNTIME_PASS ... os=ubuntu-22.04 ros=humble
ros2=/opt/ros/humble/bin/ros2
rviz2=/opt/ros/humble/bin/rviz2 gui=true
```

Calling the guard without `REQUIRED_GUI=true` and seeing `gui=false` only says the GUI checks were not requested; it does not diagnose the mounts.

DOCKER — SERIAL ACCESS

```
stat -Lc 'device=%n mode=%A uid=%u gid=%g' \
  /dev/unitree_lidar

test -c /dev/unitree_lidar
test -r /dev/unitree_lidar
test -w /dev/unitree_lidar
printf 'LIDAR_PORT=%s\n' "$LIDAR_PORT"
```

Expected `LIDAR_PORT` is `/dev/unitree_lidar`. Do not use `chmod 777`; correct the selected device and supplementary GID, then recreate the container.

1.12 Unitree driver parameters

The project-owned file is:

```
/workspace/ros2_ws/src/l1_bringup/config/unitree_l1.yaml
```

Its effective values are:

```
port                /dev/ttyUSB0 in YAML
                    overridden to /dev/unitree_lidar at launch
rotate_yaw_bias     0.0
range_scale         0.001
range_bias          0.0
range_max           50.0 m
range_min           0.0 m
cloud_frame         unilidar_lidar
cloud_topic         unilidar/cloud
cloud_scan_num      18
imu_frame           unilidar_imu
imu_topic           unilidar/imu
```

Interpretation:

- `port` selects the serial character device seen by the process.
- `rotate_yaw_bias` corrects a yaw bias before publishing.
- `range_scale` and `range_bias` convert SDK range values.
- `range_min` and `range_max` constrain published returns.
- `cloud_scan_num=18` controls SDK scan accumulation.
- frame values enter ROS message headers.
- topic values determine graph names.

The launch port precedence is:

```
explicit port:= argument
> LIDAR_PORT environment variable
> /dev/ttyUSB0 fallback
```

Project launch files and YAML are an adapter around untouched vendor code. The vendor implementation opens UART at 2,000,000 bit/s, polls the SDK, converts its cloud through PCL, and publishes PointCloud2 plus IMU using sensor timestamps. It constructs the publishers with queue depth 10.

The driver reads its parameters in its constructor. For a reproducible change:

1. edit the project YAML;
2. rebuild if required;
3. source the overlay;
4. stop and relaunch the node.

1.13 Start the L1 manually

Recommended transparent project launch

DOCKER — DRIVER + MONITOR, NO RVIZ, NO OCTOMAP

```
cd /workspace/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash

ros2 launch l1_bringup unitree_l1.launch.py \
  port:=/dev/unitree_lidar \
  cloud_topic:=unilidar/cloud \
  imu_topic:=unilidar/imu \
  cloud_frame:=unilidar_lidar \
  imu_frame:=unilidar_imu \
  monitor:=true \
  rviz:=false
```

This starts exactly the driver and read-only monitor.

Fully decomposed driver command

The vendor C++ class has the internal node-name typo `unitre_lidar_sdk_node`. The project YAML section is correctly named `unitree_lidar_ros2_node`. Direct `ros2 run` must rename the node so that YAML selection matches.

DOCKER TERMINAL A — DRIVER ONLY

```
L1_CONFIG="$(
  ros2 pkg prefix --share l1_bringup
)/config/unitree_l1.yaml"

test -r "$L1_CONFIG"

ros2 run unitree_lidar_ros2 unitree_lidar_ros2_node \
  --ros-args \
  -r __node:=unitree_lidar_ros2_node \
  --params-file "$L1_CONFIG" \
  -p port:=/dev/unitree_lidar
```

The `-r __node:=...` detail is not cosmetic: it makes the parameter-file node key apply to the direct vendor executable.

DOCKER TERMINAL B — MONITOR ONLY

```
L1_CONFIG="$(
  ros2 pkg prefix --share l1_bringup
)/config/unitree_l1.yaml"

ros2 run l1_monitor l1_monitor \
  --ros-args \
  --params-file "$L1_CONFIG"
```

The monitor subscribes without modifying data and publishes `/diagnostics`. Its project thresholds are:

```
report period      2 s
message timeout    3 s
maximum stamp age  1 s
minimum cloud rate 5 Hz
minimum IMU rate   20 Hz
window             100 messages
```

1.14 Validate real data rather than process existence

The vendor node can remain alive even if serial initialisation did not produce data. A node name and topic advertisement are not sufficient evidence.

DOCKER — GRAPH AND EFFECTIVE PORT

```
ros2 node list
ros2 topic list -t
ros2 param get /unitree_lidar_ros2_node port
```

Expected port:

```
String value is: /dev/unitree_lidar
```

DOCKER — ENDPOINTS AND QOS

```
ros2 topic info -v /unilidar/cloud
ros2 topic info -v /unilidar/imu
```

Expected interfaces:

```
/unilidar/cloud  sensor_msgs/msg/PointCloud2
/unilidar/imu    sensor_msgs/msg/Imu
```

The validated publishers were Reliable, Volatile.

DOCKER — DEMAND ONE REAL MESSAGE

```
timeout 15s ros2 topic echo \
/unilidar/cloud sensor_msgs/msg/PointCloud2 --once
```

```
timeout 15s ros2 topic echo \
/unilidar/imu sensor_msgs/msg/Imu --once
```

DOCKER — MEASURE RATES

```
ros2 topic hz /unilidar/cloud
```

Stop with Ctrl+C, then:

```
ros2 topic hz /unilidar/imu
```

DOCKER — READ DIAGNOSTICS

```
ros2 topic echo \
/diagnostics diagnostic_msgs/msg/DiagnosticArray --once
```

Real hardware validation measured:

```
cloud rate      approximately 8-10 Hz
IMU rate        approximately 210-260 Hz
points/cloud    approximately 2,000-2,140
cloud fields    x, y, z, intensity, ring, time
cloud frame     unilidar_lidar
diagnostic state stream healthy
```

These measurements are baselines, not universal guarantees.

1.15 Stop safely

In the terminal owning a foreground launch:

```
Ctrl+C once
wait for clean process shutdown
```

Then:

```
exit
```

From the host, verify:

```
docker ps \
--filter 'ancestor=unitree-l1:humble-v1.0.16' \
--format 'NAME={{.Names}} STATUS={{.Status}}'
```

Only after the driver has released the serial port should the adapter or L1 power be disconnected.

1.16 Part 1 troubleshooting

```
SYMPTOM
No stable /dev/serial/by-id entry
CAUSE
Adapter, power or cable has not enumerated.
ACTION
Check 12 V power and USB adapter; rerun check-lidar.sh.
```

```
SYMPTOM
Several serial devices found
CAUSE
Automatic selection is ambiguous.
ACTION
Set LIDAR_DEVICE to the identified stable by-id link.
```

```
SYMPTOM
Serial device is already open
CAUSE
```

An old driver/container or another process owns the tty.
ACTION
Stop the known launch cleanly. The wrapper kills nothing automatically.

SYMPTOM
/dev/unitree_lidar absent
CAUSE
Container was created without the LiDAR Compose layer.
ACTION
Exit and recreate it with --gui --lidar.

SYMPTOM
Permission denied on /dev/unitree_lidar
CAUSE
Wrong/missing numeric tty GID.
ACTION
Recompute LIDAR_GID from the resolved host tty and recreate; no chmod 777.

SYMPTOM
catkinConfig.cmake not found during colcon
CAUSE
Unrestricted discovery found Unitree's ROS 1 package.
ACTION
Build at /workspace/ros2_ws with the project colcon_defaults.yaml.

SYMPTOM
install/setup.bash: Permission denied
CAUSE
The setup script was executed.
ACTION
Use source install/setup.bash.

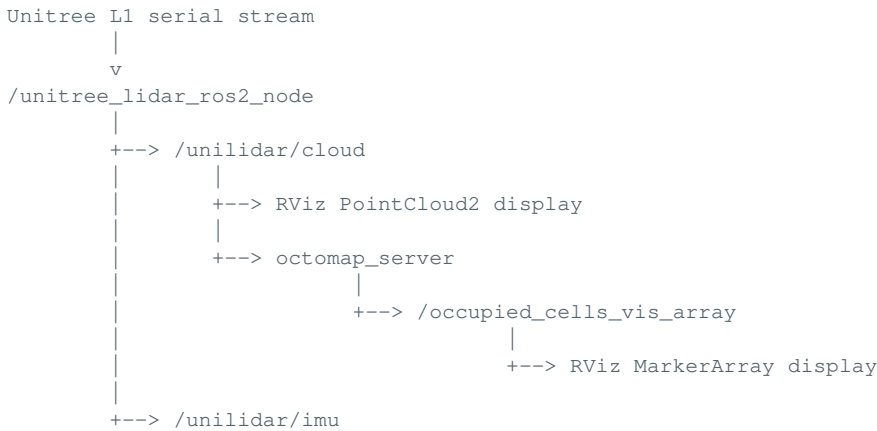
SYMPTOM
Nodes/topics exist but no messages arrive
CAUSE
Driver initialisation failed, tty is contested, or duplicate runtimes exist.
ACTION
Require echo --once, topic hz, diagnostics and exactly one publisher.

SYMPTOM
Duplicate exact node names
CAUSE
Several containers share ROS_DOMAIN_ID=42.
ACTION
Stop old runtimes; create one; attach extra shells with docker exec.

Part 2 — RViz2 configuration and use inside Docker

2.1 What RViz2 does

RViz2 is a ROS 2 subscriber and renderer. It does not communicate with the serial port, calculate a LiDAR pose, or construct an OctoMap.



RViz needs three independent conditions:

1. a publisher must produce messages;
2. an RViz display must subscribe to the exact topic with compatible QoS;
3. TF must transform the message frame into RViz's Fixed Frame.

The two project profiles deliberately use different fixed frames:

```
Raw L1 view          Fixed Frame = unilidar_lidar
OctoMap view         Fixed Frame = map
```

The raw cloud already carries frame_id=unilidar_lidar, so the raw view needs no additional TF. The map view needs map <- unilidar_lidar at each cloud timestamp.

The L1 IMU is not displayed by the supplied raw RViz profile. It is validated with ROS topic tools or consumed by a future estimator.

2.2 How a Docker RViz window reaches the desktop

rviz2 executes at:

```
/opt/ros/humble/bin/rviz2
```

inside Ubuntu 22.04 Docker. The window appears on Ubuntu 24.04 because the container receives:

```
DISPLAY
read-only Xauthority cookie
read-only X11 socket
/dev/dri
video/render group IDs
```

The chain is:

```
RViz2 process in Docker
-> OpenGL / DRI
-> X11 socket authenticated by Xauthority
-> host X11/XWayland server
-> desktop window
```

These mounts are fixed when the container is created. They cannot be added to an already-created headless container.

HOST — GUI PRE-FLIGHT, EXECUTED DURING PROJECT

```
cd /home/isr/unitree_l1_project
./scripts/gui-smoke-test.sh
```

The test creates a temporary container and checks xdpinfo, glxinfo -B and rviz2 --help. It does not start the LiDAR.

DOCKER — PROVE THAT THIS SHELL HAS GUI ACCESS

```
REQUIRE_GUI=true /workspace/scripts/assert-ros-container.sh
```

Expected marker contains:

```

DOCKER_ROS_RUNTIME_PASS
os=ubuntu-22.04
ros=humble
rviz2=/opt/ros/humble/bin/rviz2
gui=true

```

2.3 First recover a single valid runtime

The current three-container condition must be removed before assessing RViz. An open RViz window proves only rendering; it does not prove a healthy cloud.

HOST — CLEAN-STATE CHECK

```

docker ps \
  --filter 'ancestor=unitree-l1:humble-v1.0.16' \
  --format 'ID={{.ID}} NAME={{.Names}} STATUS={{.Status}}'

```

Expected before restart: no rows.

Create one runtime:

```

cd /home/isr/unitree_l1_project
./scripts/docker-shell.sh --gui --lidar

```

If inspection needs a second terminal, attach it to the single listed container:

```

docker exec -it THE_SINGLE_CONTAINER_NAME bash

```

Then:

```

cd /workspace/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash

```

2.4 Start the raw L1 view with one command

DOCKER — DRIVER + MONITOR + RAW RVIZ PROFILE

```

ros2 launch l1_bringup unitree_l1.launch.py \
  port:=/dev/unitree_lidar \
  monitor:=true \
  rviz:=true

```

This starts:

```

/unitree_lidar_ros2_node
/l1_monitor
/rviz2

```

and loads the installed unitree_l1.rviz profile. Stop all three with one Ctrl+C.

2.5 Open the raw profile manually

If the driver is already running with rviz:=false, resolve the path before starting RViz.

DOCKER — SAFE TWO-STAGE PROFILE RESOLUTION

```

RAW_SHARE="$(ros2 pkg prefix --share l1_bringup)"
RAW_CONFIG="${RAW_SHARE}/config/unitree_l1.rviz"

printf 'RAW_CONFIG=%s\n' "$RAW_CONFIG"
test -r "$RAW_CONFIG"

```

Expected:

```

/workspace/ros2_ws/install/l1_bringup/share/
l1_bringup/config/unitree_l1.rviz

```

Only after test -r succeeds:

```

rviz2 -d "$RAW_CONFIG"

```

This is safer than embedding a package lookup in the final command. A failed lookup can otherwise reduce the path to /config/... and RViz may open a default Grid-only view.

2.6 Build the raw L1 view yourself from blank RViz

DOCKER — OPEN A BLANK CONFIGURATION

Global Options

In the Displays panel:

```
Fixed Frame      unilidar_lidar
Frame Rate       30
Background Color 48; 48; 48
```

There is no leading slash in a frame ID.

Add a Grid

Choose:

```
Add -> By display type -> rviz_default_plugins/Grid
```

Set:

```
Name           Grid
Enabled         true
Alpha           0.5
Cell Size       1
Plane           XY
Plane Cell Count 20
Line Style       Lines
Line Width       0.03
Reference Frame <Fixed Frame>
Color           160; 160; 164
```

Add sensor axes

Choose:

```
Add -> rviz_default_plugins/Axes
```

Set:

```
Name           Axes
Enabled         true
Length          1
Radius          0.1
Reference Frame <Fixed Frame>
```

Add the L1 point cloud

Choose:

```
Add -> rviz_default_plugins/PointCloud2
```

Set:

```
Name           L1 Point Cloud
Enabled         true
Topic           /unilidar/cloud
Reliability Policy Reliable
Durability Policy Volatile
History Policy   Keep Last
Depth           10
Filter size      10
Position Transformer XYZ
Color Transformer AxisColor
Axis            Z
Autocompute Intensity Bounds true
Use rainbow      true
Style            Points
Size (Pixels)    3
Size (m)         0.01
Decay Time       1
Alpha           1
Use Fixed Frame  true
Selectable       true
```

Why these values:

- Reliable/Volatile matches the validated driver publisher.
- XYZ interprets the point fields as Cartesian coordinates.
- AxisColor with Z colors height, which makes geometry readable.
- one-second decay shows short recent history without indefinite smearing.
- both pixel and metric size provide visible points at useful zoom levels.

Set the camera

Choose the Orbit view:

```
Target Frame  unilidar_lidar
Distance      10
Focal Point   0, 0, 0
Pitch         0.785 rad
Yaw           0.785 rad
```

The authoritative reference is:

```
/workspace/ros2_ws/src/l1_bringup/config/unitree_l1.rviz
```

2.7 Operate and interpret RViz

With Move Camera active:

- left-drag orbits around the focal point;
- middle-drag pans;
- the mouse wheel zooms;
- Focus Camera recentres around selected geometry;
- Select inspects a rendered object;
- Measure measures spatial distance.

Useful display controls:

- the checkbox beside a display shows/hides it;
- increase Size (Pixels) if points are difficult to see;
- decrease it if nearby surfaces look too dense;
- set Decay Time=0 for only the current cloud;
- keep Decay Time=1 for approximately one second of recent scans;
- expand Status whenever an icon is yellow or red.

What a correct raw view proves:

- PointCloud2 messages reach RViz;
- the topic/QoS contract is compatible;
- the frame is resolvable;
- the rendered geometry changes with the physical scene.

What it does not prove:

- metric calibration accuracy;
- trajectory accuracy;
- SLAM;
- OctoMap correctness.

2.8 Save and reload your own RViz profile

Inside RViz choose:

```
File -> Save Config As...
```

The file dialog is inside the container. Save below the bind mount, for example:

```
/workspace/ros2_ws/src/l1_bringup/config/my_l1_view.rviz
```

The file persists on the host at:

```
/home/isr/unitree_l1_project/ros2_ws/src/
l1_bringup/config/my_l1_view.rviz
```

Reload it directly:

```
rviz2 -d \
/workspace/ros2_ws/src/l1_bringup/config/my_l1_view.rviz
```

Or use File -> Open Config.

If a new profile filename must be available through the package install share:

```
cd /workspace/ros2_ws
colcon build --packages-select l1_bringup
source install/setup.bash
```

A .rviz file stores displays, topic names, QoS, colors and camera state. It does not contain LiDAR data.

2.9 Validate the raw input independently of the GUI

Open an attached shell in the same container.

DOCKER — STRUCTURAL CHECK

```
ros2 node list | sort
ros2 topic list -t | sort
ros2 topic info -v /unilidar/cloud
```

There must be exactly one cloud publisher.

DOCKER — TEMPORAL CHECK

```
ros2 topic hz /unilidar/cloud
```

The observed baseline was roughly 8-10 Hz. Stop the measurement with Ctrl+C.

DOCKER — FRAME CHECK

```
ros2 topic echo \
/unilidar/cloud \
sensor_msgs/msg/PointCloud2 \
--field header \
--once
```

Expected:

```
frame_id: unilidar_lidar
```

If these checks fail, changing RViz colors or camera controls cannot repair the underlying graph.

2.10 Build the OctoMap view yourself from blank RViz

This view is used only when octomap_server and the required TF are running.

Global Options

```
Fixed Frame      map
Frame Rate       30
Background Color 48; 48; 48
```

Use the same Grid and Axes values as the raw view.

Raw cloud display in map

Add PointCloud2:

```
Name          L1 Point Cloud
Topic          /unilidar/cloud
Reliability Policy Best Effort
Durability Policy Volatile
History Policy  Keep Last
Depth          10
Filter size    10
Position       XYZ
Color Transformer AxisColor
Axis           Z
Style          Points
Size (Pixels)  3
Size (m)       0.01
Decay Time     1
Alpha         1
```

A Best Effort subscriber is compatible with the current Reliable publisher and is appropriate for high-rate visual sensor data.

Occupied voxel display

Add rviz_default_plugins/MarkerArray:

```
Name          Occupied OctoMap Voxels
Topic          /occupied_cells_vis_array
Queue Size     100
Reliability Policy Reliable
Durability Policy Transient Local
History Policy  Keep Last
Depth          1
Enabled        true
```

Transient Local is important because latch=true makes OctoMap retain its latest map output for a late subscriber.

Camera

```
View          Orbit
Target Frame  map
Distance      12
Focal Point   0, 0, 0
Pitch         0.785 rad
Yaw           0.785 rad
```

The authoritative reference profile is:

```
/workspace/ros2_ws/src/l1_octomap_bringup/
config/l1_octomap.rviz
```

2.11 Open the OctoMap profile manually

DOCKER — RESOLVE, TEST, THEN OPEN

```
OCTOMAP_SHARE="$(
  ros2 pkg prefix --share l1_octomap_bringup
)"
OCTOMAP_RVIZ="$(OCTOMAP_SHARE)/config/l1_octomap.rviz"

printf 'OCTOMAP_RVIZ=%s\n' "$OCTOMAP_RVIZ"
test -r "$OCTOMAP_RVIZ"
rviz2 -d "$OCTOMAP_RVIZ"
```

The package spelling is exactly:

```
l1_octomap_bringup
```

not:

```
l1_octomap_bingup
```

2.12 Start the complete mapped view

DOCKER — ONE DRIVER, ONE MAP SERVER, ONE RVIZ

```
ros2 launch l1_octomap_bringup \
  unitree_l1_octomap.launch.py \
  port:=/dev/unitree_lidar \
  static_sensor:=true \
  rviz:=true
```

The corrected combined launch:

- starts the L1 child with its own rviz=false;
- starts the OctoMap child with its own rviz=false;
- isolates both child argument contexts with GroupAction(scoped=True);
- keeps the parent's public rviz:=true;
- starts exactly one RViz with the combined profile.

The regression test verifies child values false,false, parent value true, and final parent context true. Nine package tests passed, and a headless launch probe found exactly one RViz process using the installed profile.

For a stationary sensor, the command also supplies an identity transform:

```
map -> unilidar_lidar
translation  0, 0, 0
rotation    0, 0, 0
```

This is bench-only. Part 3 explains why it is invalid during motion.

2.13 RViz troubleshooting by layer

```
SYMPTOM
rviz: command not found
MEANING
The ROS 2 executable is rviz2.
ACTION
Use rviz2 after sourcing Humble and the overlay.
```

```
SYMPTOM
rviz2: Missing value after '-d'
MEANING
-d requires an immediately following readable filename.
ACTION
Resolve the path into a variable, test -r, then execute.
```

```
SYMPTOM
```

Package not found
 MEANING
 Overlay is not sourced or package spelling is wrong.
 ACTION
 source install/setup.bash; use bringup, not bingup.

SYMPTOM
 RViz shows only Global Options and Grid
 MEANING
 The default profile opened; the intended profile did not.
 ACTION
 Resolve the package share separately and test the .rviz path.

SYMPTOM
 The attempted path is /config/l1_octomap.rviz
 MEANING
 Package substitution returned an empty prefix.
 ACTION
 Stop RViz; correct/source package lookup before forming the path.

SYMPTOM
 XDG_RUNTIME_DIR not set
 MEANING
 Qt falls back to /tmp/runtime-ros.
 ACTION
 Usually harmless if the RViz window opens.

SYMPTOM
 Stereo is NOT SUPPORTED
 MEANING
 The OpenGL context lacks stereo buffers.
 ACTION
 Harmless for this project.

SYMPTOM
 Could not connect to display
 MEANING
 The container lacks valid X11 transport.
 ACTION
 Exit and recreate it with docker-shell.sh --gui --lidar.

SYMPTOM
 Fixed Frame is red
 MEANING
 The selected frame is absent or cannot transform incoming data.
 ACTION
 Use unilidar_lidar for raw view; map for mapping; verify TF.

SYMPTOM
 PointCloud2 says no messages
 MEANING
 No live publisher, wrong topic/QoS, or duplicate runtimes.
 ACTION
 Use topic info -v and topic hz; require exactly one publisher.

SYMPTOM
 Message Filter queue is full
 MEANING
 Clouds cannot be transformed into map at their timestamps.
 ACTION
 Repair map <- unilidar_lidar TF; this is not a rendering fault.

SYMPTOM
 Only /rosout and /parameter_events exist
 MEANING
 No application graph is running in that domain.
 ACTION
 Start the L1 launch and check ROS_DOMAIN_ID=42.

Part 3 — OctoMap setup, configuration and operation

3.1 What OctoMap calculates

octomap_server consumes timestamped PointCloud2 scans and the TF transform that locates each scan in a world frame. For every valid return it ray-casts from the sensor origin:

```
sensor origin ---- free traversed voxels ----> occupied endpoint
```

Repeated measurements update probabilistic occupancy in an octree:

- a hit raises endpoint occupancy;
- a miss lowers occupancy along the free ray;
- probability clamps prevent unlimited confidence;
- the octree stores fine detail only where required;
- lossless pruning compresses identical child states.

OctoMap is a 3-D occupancy mapper. It is not a pose estimator.

3.2 Stationary mapping versus SLAM

For a completely stationary bench sensor, the project may define:

```
map -> unilidar_lidar = identity transform
```

This says the LiDAR origin and axes define the map origin and axes. All scans are inserted from one fixed pose.

For a moving robot, the required model is different:

```
map -> odom -> base_link -> unilidar_lidar
```

- a localisation/odometry/SLAM system supplies the time-varying pose;
- a calibrated static transform supplies the sensor mounting geometry;
- TF must be valid at each cloud timestamp;
- static_sensor:=false disables the bench identity transform.

Keeping the identity world-to-LiDAR transform while moving produces smeared or doubled surfaces. It is not SLAM, even when the result looks like a large map. No mobile pose estimator has yet been selected or validated in this project. The proven result is stationary OctoMap occupancy mapping.

3.3 OctoMap dependency and project adapter

The project builds the ROS 2 branch of:

```
https://github.com/OctoMap/octomap_mapping.git
tag      2.3.1
commit   f79da9a9a1fcdf82e72dab4df288d6cc27c6e163
```

It is pinned in /workspace/config/dependencies.repos.

The project-owned l1_octomap_bringsup package adds:

```
config/octomap.yaml
config/l1_octomap.rviz
config/saved_octomap.rviz
launch/l1_octomap.launch.py
launch/unitree_l1_octomap.launch.py
launch/view_saved_octomap.launch.py
tests for launch/config contracts
```

The upstream Unitree and OctoMap repositories remain unmodified.

DOCKER — VERIFY INSTALLED PACKAGES

```
ros2 pkg prefix --share octomap_server
ros2 pkg prefix --share l1_octomap_bringsup
ros2 pkg xml octomap_server | sed -n '1,12p'
```

If the pinned checkout must be restored:

```
cd /workspace/ros2_ws
vcs import --skip-existing src \
  < /workspace/config/dependencies.repos

git -C src/octomap_mapping rev-parse HEAD
```

```
git -C src/octomap_mapping describe --tags --exact-match HEAD
```

```
colcon build
source install/setup.bash
```

3.4 ROS graph and data contracts

```
/dev/unitree_lidar
|
v
/unitree_lidar_ros2_node
|
| /unilidar/cloud
| sensor_msgs/msg/PointCloud2
| header.frame_id = unilidar_lidar
|
v
/octomap_server input "cloud_in"
|
| requires map <- unilidar_lidar at cloud time
|
+-- /occupied_cells_vis_array
+-- /free_cells_vis_array
+-- /octomap_binary
+-- /octomap_full
+-- /octomap_point_cloud_centers
+-- /projected_map
+-- /octomap_binary service
+-- /octomap_full service
+-- /octomap_server/reset service
+-- /octomap_server/clear_bbox service
```

The project remaps:

```
octomap_server cloud_in <- /unilidar/cloud
```

The upstream cloud subscriber uses sensor-data QoS. With project latch=true, map outputs use transient-local durability so late subscribers can receive the current map.

Expected combined stationary nodes:

```
/unitree_lidar_ros2_node
/l1_monitor
/l1_static_lidar_transform
/octomap_server
/rviz2                                when rviz:=true
```

An internal TF listener node may also appear.

3.5 Exact OctoMap configuration

The project file is:

```
/workspace/ros2_ws/src/l1_octomap_bringup/config/octomap.yaml
```

Effective settings:

frame_id	map
base_frame_id	unilidar_lidar
resolution	0.10
sensor_model.max_range	15.0
sensor_model.hit	0.7
sensor_model.miss	0.4
sensor_model.min	0.12
sensor_model.max	0.97
filter_ground_plane	false
filter_speckles	false
compress_map	true
incremental_2D_projection	false
publish_free_space	false
latch	true

Frame parameters

frame_id=map is the world frame in which the octree is accumulated and published.

base_frame_id=unilidar_lidar describes the sensor/base frame used by the server in this stationary setup.

Resolution

resolution=0.10 means the finest voxel has a 10 cm edge.

smaller resolution more detail, more CPU, memory and output size
larger resolution less detail, lower cost, stronger spatial quantisation

Resolution is chosen when the octree is constructed. Change it before launch, then restart the server and create a separate result file.

Range

sensor_model.max_range=15.0 caps useful ray length at 15 m. Returns within range create occupied endpoints; longer measurements clear only to the configured limit.

The driver can publish farther returns (range_max=50.0), but mapping is deliberately restricted to 15 m to control cost and distant noise. These are different layers and should not be confused.

Occupancy update probabilities

```
hit    0.7   evidence that an endpoint is occupied
miss   0.4   evidence that a traversed voxel is free
min    0.12  lower occupancy-probability clamp
max    0.97  upper occupancy-probability clamp
```

Internally OctoMap stores log-odds. Repeated consistent observations drive a voxel toward a clamp; contradictory observations can reverse confidence. These values are initial engineering choices, not universal optima.

Filters and output behaviour

filter_ground_plane=false inserts ground returns normally; no RANSAC ground removal is performed.

filter_speckles=false preserves isolated occupied voxels.

compress_map=true enables lossless octree pruning after insertion.

incremental_2D_projection=false avoids the incremental 2-D projection mode.

publish_free_space=false keeps free-space occupancy internally but does not request its potentially heavy marker visualisation.

latch=true selects retained/transient-local map publication.

For controlled experiments, change versioned YAML or explicit launch arguments, stop, restart and save under a new filename. Do not tune several variables and overwrite the same map.

3.6 Prepare every manual component terminal

All following terminals are shells in the same container.

EVERY DOCKER TERMINAL

```
cd /workspace/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
```

Before the experiment:

```
REQUIRED_GUI=true /workspace/scripts/assert-ros-container.sh
test -c /dev/unitree_lidar
test -r /dev/unitree_lidar
test -w /dev/unitree_lidar
```

Resolve configuration paths and refuse missing files:

```
L1_CONFIG="$(
  ros2 pkg prefix --share ll_bringup
)/config/unitree_ll.yaml"

OCTOMAP_CONFIG="$(
  ros2 pkg prefix --share ll_octomap_bringup
)/config/octomap.yaml"

OCTOMAP_RVIZ="$(
  ros2 pkg prefix --share ll_octomap_bringup
)/config/ll_octomap.rviz"

test -r "$L1_CONFIG"
test -r "$OCTOMAP_CONFIG"
test -r "$OCTOMAP_RVIZ"
```

3.7 Fully manual stationary bench experiment

This is the decomposed procedure. It exposes every process and is the best way to understand the stack.

DOCKER TERMINAL A — UNITREE DRIVER

```
ros2 run unitree_lidar_ros2 unitree_lidar_ros2_node \  
  --ros-args \  
  -r __node:=unitree_lidar_ros2_node \  
  --params-file "$L1_CONFIG" \  
  -p port:=/dev/unitree_lidar \  
  -p cloud_topic:=/unilidar/cloud \  
  -p imu_topic:=/unilidar/imu \  
  -p cloud_frame:=unilidar_lidar \  
  -p imu_frame:=unilidar_imu
```

Only this one process should own the serial port.

DOCKER TERMINAL B — OPTIONAL HEALTH MONITOR

```
ros2 run l1_monitor l1_monitor \  
  --ros-args \  
  --params-file "$L1_CONFIG"
```

DOCKER TERMINAL C — STATIONARY IDENTITY TF

Keep the L1 completely immobile:

```
ros2 run tf2_ros static_transform_publisher \  
  --x 0 --y 0 --z 0 \  
  --roll 0 --pitch 0 --yaw 0 \  
  --frame-id map \  
  --child-frame-id unilidar_lidar
```

This declares map coordinates equal to LiDAR coordinates. Do not run it on a moving platform.

DOCKER TERMINAL D — OCTOMAP SERVER

```
ros2 run octomap_server octomap_server_node \  
  --ros-args \  
  --params-file "$OCTOMAP_CONFIG" \  
  -r cloud_in:=/unilidar/cloud
```

This command loads the project YAML and remaps upstream input cloud_in.

DOCKER TERMINAL E — RVIZ MAPPED VIEW

```
test -r "$OCTOMAP_RVIZ" && \  
  rviz2 -d "$OCTOMAP_RVIZ"
```

The view overlays:

- current L1 PointCloud2 data;
- accumulated occupied OctoMap voxel markers;
- map axes and reference grid.

3.8 Launch-file alternatives after understanding the components

Mapping only, when one healthy driver already exists

```
ros2 launch l1_octomap_bringup l1_octomap.launch.py \  
  cloud_topic:=/unilidar/cloud \  
  world_frame:=map \  
  lidar_frame:=unilidar_lidar \  
  resolution:=0.10 \  
  max_range:=15.0 \  
  static_sensor:=true \  
  rviz:=true
```

This starts the bench TF, octomap_server and RViz. It does not start the Unitree driver.

Complete combined graph

```
ros2 launch l1_octomap_bringup \  
  unitree_l1_octomap.launch.py \  
  port:=/dev/unitree_lidar \  
  cloud_topic:=/unilidar/cloud \  
  world_frame:=map \  
  lidar_frame:=unilidar_lidar \  
  resolution:=0.10 \  
  max_range:=15.0 \  
  static_sensor:=true \  
  rviz:=true
```

This starts:

Unitree L1 engineering manual

1. Unitree driver;
2. read-only monitor;
3. stationary identity TF;
4. octomap_server;
5. exactly one combined-profile RViz process.

Inspect arguments without launching:

```
ros2 launch l1_octomap_bringup \
  unitree_l1_octomap.launch.py --show-args
```

3.9 Inspect graph, cloud, TF, QoS and parameters

Use an attached shell in the same container.

DOCKER — GRAPH

```
ros2 node list | sort
ros2 topic list -t | sort
ros2 service list -t | sort
```

DOCKER — CLOUD

```
ros2 topic info --verbose /unilidar/cloud
ros2 topic hz /unilidar/cloud
```

Stop the rate command with Ctrl+C.

```
ros2 topic echo \
  /unilidar/cloud \
  sensor_msgs/msg/PointCloud2 \
  --field header \
  --once
```

DOCKER — TF

```
ros2 run tf2_ros tf2_echo \
  map unilidar_lidar
```

For the stationary identity case, translation and rotation should be zero. Stop tf2_echo with Ctrl+C.

DOCKER — OCTOMAP OUTPUT ENDPOINTS

```
ros2 topic info --verbose \
  /occupied_cells_vis_array
```

```
ros2 topic info --verbose \
  /octomap_binary
```

```
ros2 topic hz \
  /occupied_cells_vis_array
```

DOCKER — REQUIRE NON-EMPTY OCCUPIED MARKERS

```
ros2 topic echo \
  /occupied_cells_vis_array \
  visualization_msgs/msg/MarkerArray \
  --qos-reliability reliable \
  --qos-durability transient_local \
  --filter \
  'len(m.markers) > 0 and any(len(x.points) > 0 for x in m.markers)' \
  --truncate-length 1 \
  --once
```

DOCKER — INSPECT BINARY MAP METADATA

```
ros2 topic echo \
  /octomap_binary \
  octomap_msgs/msg/Octomap \
  --qos-reliability reliable \
  --qos-durability transient_local \
  --truncate-length 8 \
  --once
```

DOCKER — EFFECTIVE PARAMETERS

```
ros2 param list /octomap_server
ros2 param get /octomap_server frame_id
ros2 param get /octomap_server base_frame_id
ros2 param get /octomap_server resolution
ros2 param get /octomap_server sensor_model.max_range
ros2 param dump /octomap_server
```

Real stationary validation observed:

cloud	approximately 9.6–9.8 Hz
OctoMap markers	approximately 7–8.3 Hz
marker array	17 markers
occupied markers	2
occupied points	13,247

These values demonstrate one successful session, not required exact counts.

3.10 Evaluate map health

A useful stationary occupancy-map assessment requires:

- exactly one driver and one octomap_server;
- nonzero cloud rate;
- valid timestamped TF;
- nonempty /octomap_binary payload;
- occupied marker points;
- expected frame and 0.10 m resolution;
- a nonempty saved map;
- visually sharp, single surfaces;
- no large unexplained isolated clusters;
- repeatability between controlled captures.

The project evaluator checks structural map health with matching transient-local QoS:

HOST — OPTIONAL AUTOMATED EVALUATOR

```
cd /home/isr/unitree_l1_project
./scripts/evaluate-octomap.sh
```

The wrapper uses docker exec; its ROS/Python check is visible in the script. Its pass marker is:

```
OCTOMAP_MAPPING_HEALTH_PASS
```

This does not measure trajectory accuracy and must not be reported as SLAM accuracy.

3.11 Save the map with the official OctoMap command

Keep octomap_server running and use an attached shell.

DOCKER — SAVE A BINARY TREE

```
mkdir -p /workspace/maps

MAP=/workspace/maps/my_room_01.bt
test ! -e "$MAP"

ros2 service list | grep -Fx /octomap_binary

ros2 run octomap_server octomap_saver_node \
  --ros-args \
  -p octomap_path:="$MAP"

test -s "$MAP"
stat -c 'file=%n bytes=%s modified=%y' "$MAP"
sha256sum "$MAP"
```

This is the upstream README command with a concrete persistent path. The host file appears at:

```
/home/isr/unitree_l1_project/maps/my_room_01.bt
```

File types:

.bt	compact binary occupancy tree
.ot	full OctoMap serialisation

For an explicit full map:

DOCKER — SAVE A FULL TREE

```
ros2 run octomap_server octomap_saver_node \  
  --ros-args \  
  -p full:=true \  
  -p octomap_path:=/workspace/maps/my_room_01.ot
```

Validated project map examples:

```
l1_real_bench_20260716.bt  
  stored nodes 45,149  
  resolution   0.1 m  
  bytes        18,902  
  
l1_bench_map_20260716.bt  
  stored nodes 58,202  
  resolution   0.1 m  
  bytes        23,276
```

Binary writing and pruning can make stored header counts differ from a live message count.

3.12 Inspect and reopen a saved map

After saving, stop the live graph with one Ctrl+C.

DOCKER — FILE-LEVEL CHECK

```
head -n 7 /workspace/maps/my_room_01.bt  
stat -c 'bytes=%s modified=%y' \  
  /workspace/maps/my_room_01.bt  
sha256sum /workspace/maps/my_room_01.bt
```

Expected header fields include:

```
id OcTree  
size <positive integer>  
res 0.1  
data
```

DOCKER — PROJECT SAVED-MAP LAUNCH

```
ros2 launch l1_octomap_bringup \  
  view_saved_octomap.launch.py \  
  map_path:=/workspace/maps/my_room_01.bt \  
  frame_id:=map \  
  rviz:=true
```

This does not need the physical L1.

Fully decomposed saved-map view

DOCKER TERMINAL A — LOAD THE FILE

```
ros2 run octomap_server octomap_server_node \  
  --ros-args \  
  --params-file "$OCTOMAP_CONFIG" \  
  -p octomap_path:=/workspace/maps/my_room_01.bt \  
  -p frame_id:=map \  
  -p base_frame_id:=map
```

DOCKER TERMINAL B — SAVED-MAP RVIZ PROFILE

```
SAVED_RVIZ="$(  
  ros2 pkg prefix --share l1_octomap_bringup  
) /config/saved_octomap.rviz"  
  
test -r "$SAVED_RVIZ" && \  
  rviz2 -d "$SAVED_RVIZ"
```

The saved-map profile shows the retained MarkerArray without the live L1 cloud.

3.13 Reset or clear the in-memory map

These commands mutate the current in-memory map. Save first if its state matters. Saved files are not changed.

DOCKER — RESET THE WHOLE LIVE MAP

```
ros2 service call \  
  /octomap_server/reset \  
  std_srvs/srv/Empty \  
  "{}"
```

DOCKER — CLEAR AN AXIS-ALIGNED BOX IN MAP COORDINATES

```
ros2 service call \  
  /octomap_server/clear_bbox \  
  octomap_msgs/srv/BoundingBoxQuery \  
  "{min: {x: -1.0, y: -1.0, z: 0.0}, \  
max: {x: 1.0, y: 1.0, z: 2.0}}"
```

3.14 Deliberate parameter experiments

Change one variable per experiment and save a new file.

Resolution experiment

```
ros2 launch l1_octomap_bringup \  
  unitree_l1_octomap.launch.py \  
  resolution:=0.05 \  
  max_range:=15.0 \  
  static_sensor:=true \  
  rviz:=true
```

Compare against 0.10 m:

- map byte size;
- CPU/memory;
- occupied point count;
- edge/detail quality;
- isolated noise.

Range experiment

```
ros2 launch l1_octomap_bringup \  
  unitree_l1_octomap.launch.py \  
  resolution:=0.10 \  
  max_range:=10.0 \  
  static_sensor:=true \  
  rviz:=true
```

Compare distant noise and completeness. Do not infer improvement from file size alone.

Probability/filter experiment

Edit project-owned config/octomap.yaml, record the old/new values, rebuild the package if necessary, source the overlay, relaunch and save a new map. Do not modify upstream octomap_mapping.

3.15 Moving-platform command and missing prerequisite

DOCKER — DISABLE BENCH TF

```
ros2 launch l1_octomap_bringup \  
  unitree_l1_octomap.launch.py \  
  port:=/dev/unitree_lidar \  
  static_sensor:=false \  
  rviz:=true
```

This command is intentionally incomplete by itself. An external estimator must already supply timestamp-valid TF to unilidar_lidar.

Check:

```
ros2 run tf2_ros tf2_echo \  
  map unilidar_lidar
```

```
ros2 topic hz /unilidar/cloud
```

For actual SLAM evaluation, record at least:

```
/unilidar/cloud  
/unilidar/imu  
/tf  
/tf_static  
external odometry or pose topic  
ground-truth trajectory when available
```

Then assess:

- Absolute Trajectory Error;
- Relative Pose Error;
- translation/rotation drift per distance;

- loop-closure consistency;
- scan overlap and wall sharpness;
- map completeness;
- repeatability across equivalent runs.

OctoMap map-health metrics do not replace these trajectory metrics.

3.16 Part 3 troubleshooting

SYMPTOM

Package `ll_octomap_bringup` not found

CAUSE

Overlay not sourced or `bringup` misspelled.

ACTION

source `install/setup.bash`; use `ros2 pkg prefix --share`.

SYMPTOM

Marker topic exists but remains empty

CAUSE

No fresh cloud or no valid TF at cloud timestamps.

ACTION

Check cloud Hz and `tf2_echo` map `unilidar_lidar`.

SYMPTOM

Message Filter discarding because queue is full

CAUSE

TF filter cannot transform queued clouds into map.

ACTION

Supply correct timestamped TF; do not enlarge queues first.

SYMPTOM

Doubled or smeared walls

CAUSE

Sensor moved while static identity TF was active.

ACTION

Stop; use `static_sensor:=false` and an external pose estimator.

SYMPTOM

Nothing to publish, octree is empty

CAUSE

No scan has been inserted yet.

ACTION

Briefly normal at startup; if persistent, verify cloud and TF.

SYMPTOM

Could not open file with an empty live map path

CAUSE

Upstream server checked the default empty `octomap_path`.

ACTION

Benign in live-building mode if cloud insertion then succeeds.

SYMPTOM

Cloud is 0 Hz or serial times out

CAUSE

Duplicate driver/container or stale tty ownership.

ACTION

One LiDAR, one driver, one container; clean restart.

SYMPTOM

Saved map is empty

CAUSE

Saver ran before a nonempty map or contacted wrong domain.

ACTION

Verify `/octomap_binary` service and nonempty map output first.

SYMPTOM

Late RViz receives no current markers

CAUSE

Marker QoS does not match retained output.

ACTION

Use Reliable and Transient Local for `MarkerArray`.

3.17 Complete manual laboratory sequence

The following is the shortest sequence that remains transparent.

HOST

```
cd /home/isr/unitree_l1_project
./scripts/check-lidar.sh
./scripts/gui-smoke-test.sh
./scripts/docker-shell.sh --gui --lidar
```

DOCKER — PRE-FLIGHT

```
cd /workspace/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
REQUIRED_GUI=true /workspace/scripts/assert-ros-container.sh
test -c /dev/unitree_lidar
```

DOCKER — RAW L1 VIEW

```
ros2 launch l1_bringup unitree_l1.launch.py \
  port:=/dev/unitree_lidar \
  monitor:=true \
  rviz:=true
```

Stop with Ctrl+C after validating the raw view.

DOCKER — STATIONARY OCTOMAP VIEW

```
ros2 launch l1_octomap_bringup \
  unitree_l1_octomap.launch.py \
  port:=/dev/unitree_lidar \
  resolution:=0.10 \
  max_range:=15.0 \
  static_sensor:=true \
  rviz:=true
```

ATTACHED DOCKER SHELL — CHECK AND SAVE

```
ros2 topic hz /unilidar/cloud
ros2 run tf2_ros tf2_echo map unilidar_lidar
ros2 topic info --verbose /occupied_cells_vis_array

mkdir -p /workspace/maps
ros2 run octomap_server octomap_saver_node \
  --ros-args \
  -p octomap_path:=/workspace/maps/my_room_01.bt

test -s /workspace/maps/my_room_01.bt
```

Stop the combined launch with Ctrl+C.

DOCKER — REOPEN WITHOUT THE L1

```
ros2 launch l1_octomap_bringup \
  view_saved_octomap.launch.py \
  map_path:=/workspace/maps/my_room_01.bt \
  frame_id:=map \
  rviz:=true
```

This sequence demonstrates, separately and visibly:

1. container/device/GUI correctness;
2. live PointCloud2 acquisition;
3. RViz rendering;
4. TF correctness;
5. OctoMap accumulation;
6. official .bt saving;
7. persistent saved-map reopening.

3.18 Evidence and limitations

The project evidence is split by purpose:

```
docs/configuration-log.md
  exact chronology, commands, errors, corrections and validation

docs/versions-lock.md
  pinned and observed component identities

docs/decisions.md
  architectural decisions and rejected unsafe alternatives
```

```
docs/validation-matrix.md
  test IDs, methods, results and current verdicts

docs/hardware-runbook.md
  electrical and operational safety

logs/tests
  raw validation logs

maps
  generated .bt/.ot evidence, normally outside source Git history
```

Validated achievements:

- real L1 PointCloud2 and IMU acquisition;
- healthy measured streams;
- Docker-only RViz/OpenGL operation;
- recorded/replayed sensor data;
- nonempty stationary OctoMap generation;
- official map saving and saved-map reopening;
- corrected one-RViz combined launch with automated regression coverage.

Remaining boundary:

- the live graph must be revalidated after the user closes the currently duplicated containers;
- mobile pose estimation and trajectory accuracy are not implemented;
- stationary OctoMap must not be relabelled as mobile SLAM.